Get the Response of a PUT API Request

Updating a Booking with PUT in Postman

Hands-On	8
Scenario	Get the response of PUT API Request
Tool	Postman (Desktop or Web)
Method	PUT
Endpoint URL	https://restful-booker.herokuapp.com/booking/:id
Auth	Basic Auth (admin / password123)
Duration	20 minutes
Prerequisite	An existing bookingid from a previous POST /booking call

Objective

The objective of this assignment is to send a **PUT** request from Postman to fully replace the contents of an existing booking on the Restful Booker API. The request uses **Basic Auth** (with the documented admin credentials) for authorization, sets the booking id as a path parameter, and supplies the new booking details as a JSON body. The server returns the updated booking when the request succeeds.

PUT vs POST — What is the difference?

POST creates a new resource — the server assigns an id and returns it in the response. **PUT** updates an existing resource by completely replacing it with the payload supplied in the request body. Because PUT replaces the whole record, every field that the booking should contain after the update **must** be provided. Missing fields are typically rejected as a **400 Bad Request**.

Note on Authorization

> The Restful Booker API accepts two equivalent ways to authorize a PUT: a **Cookie** header containing a token from `/auth`, or **HTTP Basic Auth** with the built-in admin credentials. This assignment uses Basic Auth with username **admin** and password **password123**. Postman automatically encodes these to Base64 and adds an `Authorization: Basic <...>` header to the outgoing request.

Steps to Send the PUT Request

1 Go to the Right Workspace

Open Postman (desktop or web) and sign in if not already logged in.

Click the **Workspaces** dropdown in the upper-left corner.

Select the workspace used in the previous assignments.

2 Go to the Right Collection

In the left sidebar, click the **Collections** tab.

Open the collection that contains the previous booking requests.

3 Add a New Request

Click the **Add a request** link inside the collection, or use the three-dot (**...**) menu next to the collection name and choose **Add request**.

4 New Tab Opens with the Empty Request

Postman opens a new request tab in the central panel with method **GET** and an empty URL by default.

5 Give the Request a Meaningful Name

Rename the request to something descriptive such as *Update Booking* or *PUT Restful Booker Booking*.

Save with **Ctrl + S** / **Cmd + S**.

6 Set the HTTP Method to PUT

Click the **method dropdown** on the left of the URL field.

Select **PUT** from the list. PUT is used to **fully replace** an existing resource on the server.

7 Configure Basic Auth

Click the **Authorization** tab below the URL bar.

From the **Type** dropdown, choose **Basic Auth**.

Enter the credentials in the right-hand panel:

- **Username**: `admin`
- **Password**: `password123`

Postman will automatically build a header in the form `Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=` and attach it to every send.

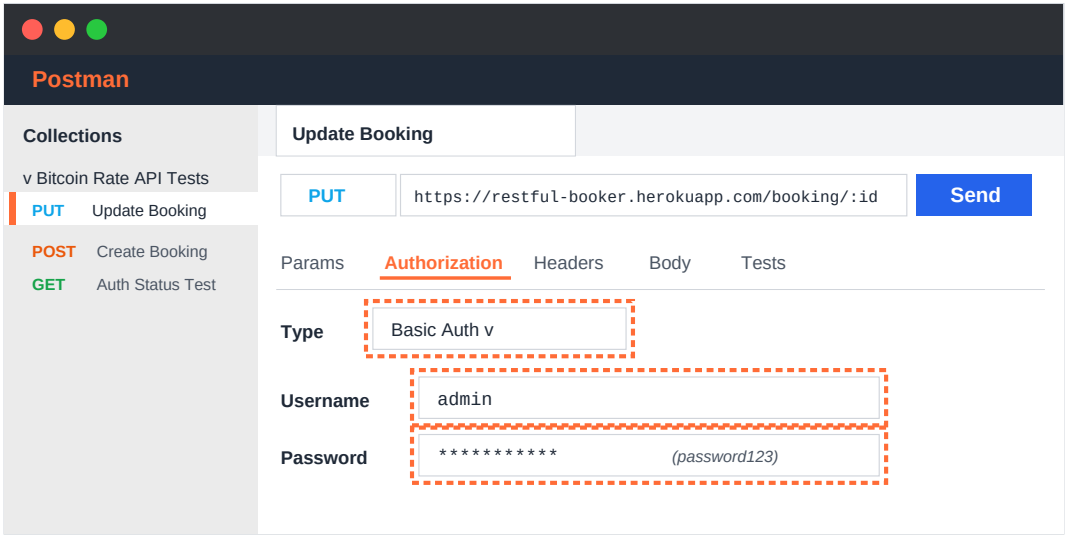

*Figure 1 — **Authorization** tab with type set to **Basic Auth**, and the admin credentials filled in.*

8 Set the Request URL with Path Parameter

Click into the URL field on the request line and enter:

`https://restful-booker.herokuapp.com/booking/:id`

The **:id** portion is a Postman **path variable**. As soon as it is typed, Postman recognizes it and exposes a **Path Variables** table under the **Params** tab.

Open the **Params** tab and set the **id** path variable to the bookingid returned from the earlier POST /booking call (for example *4271*).

Alternative: the URL can also be typed directly with the id, e.g. `.../booking/4271`, but using **:id** as a path variable is cleaner and easier to reuse.

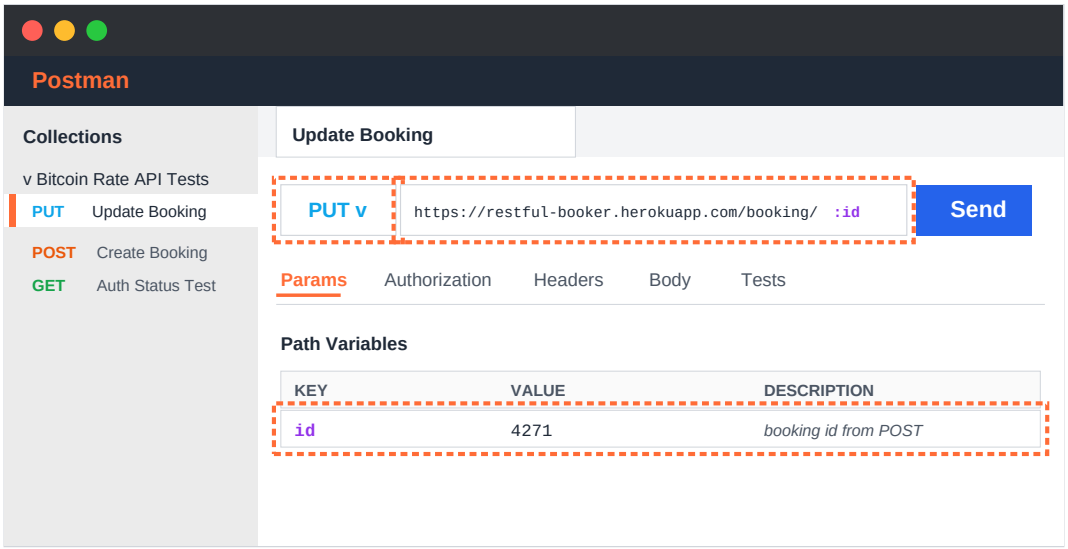

*Figure 2 — PUT URL with the **:id** path parameter and the Path Variables row filled in with the actual booking id under the **Params** tab.*

9

Add the Updated Booking Body

Click the **Body** tab.

Choose the **raw** radio option and select **JSON** from the format dropdown on the right (Postman will set the `Content-Type: application/json` header).

Paste the updated booking JSON shown in the **Sample Request Body** section below into the editor. Remember — PUT **replaces** the booking, so all fields must be present.

Sample Request Body (Input)

If no input file is provided, use this updated payload as the request body. The example changes the first name, total price, deposit flag, dates and additional needs:

```
{
  "firstname" : "James",
  "lastname"  : "Brown",
  "totalprice" : 200,
  "depositpaid" : false,
  "bookingdates" : {
    "checkin"  : "2024-02-01",
    "checkout" : "2024-02-10"
  },
  "additionalneeds" : "Lunch"
}
```

10

Click the “Send” Button and Verify the Response

Click the blue **Send** button on the right side of the URL bar.

Postman dispatches the PUT request and displays the server's response in the lower **Response** panel.

Verify the following:

• **Status** — should be **200 OK** in green.
• **Body** — contains the updated booking object with all the new field values that were sent.
• **Time / Size** — the request typically completes well under a second.

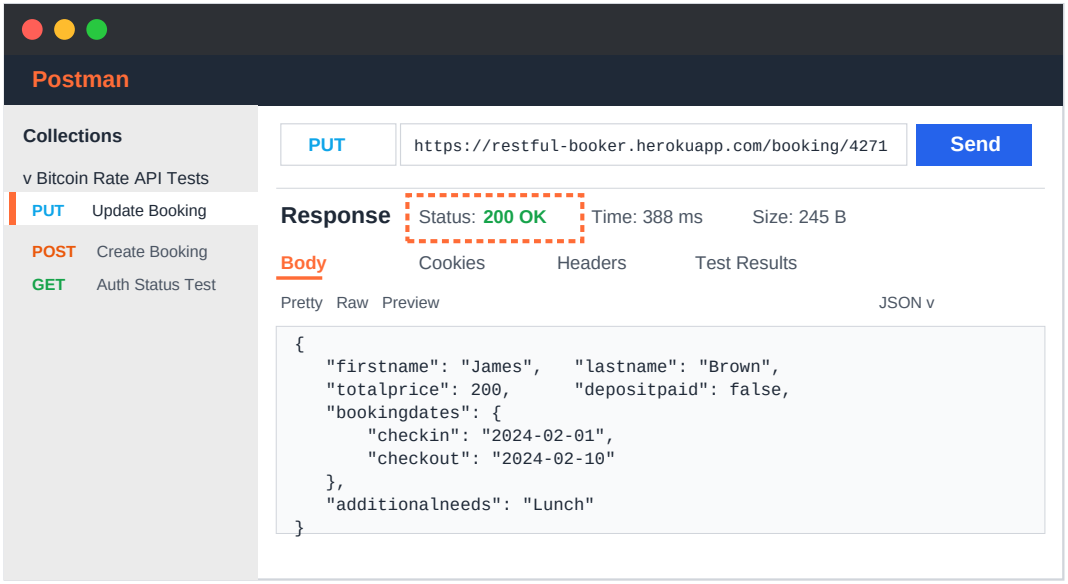

*Figure 3 — Response panel with status **200 OK** and the updated booking returned by the server.*

Expected Response

On success, the server returns **HTTP 200 OK** with a JSON payload containing the updated booking (note: there is no *bookingid* wrapper on PUT — the booking object is returned directly):

```
{
  "firstname": "James",
  "lastname": "Brown",
  "totalprice": 200,
  "depositpaid": false,
  "bookingdates": {
    "checkin": "2024-02-01",
    "checkout": "2024-02-10"
  },
  "additionalneeds": "Lunch"
}
```

*If the response shows **403 Forbidden** or **401 Unauthorized**, double-check the Basic Auth credentials. If it shows **405 Method Not Allowed**, the bookingid in the URL does not exist on the server (remember, the API resets every 10 minutes — the booking may have been wiped). Re-create it with a fresh POST and use the new id.*

Verification Checklist

1. Status code is **200 OK**.
2. Response body matches the data sent in the request body (firstname, lastname, dates, etc.).
3. Basic Auth is selected under **Authorization** with the correct username and password.
4. The URL uses `:id` and the Params tab shows the bookingid under **Path Variables**.
5. Request is saved (**Ctrl + S** / **Cmd + S**) inside the collection.

Conclusion

A PUT request to update an existing booking has been successfully sent and the response verified. This exercise introduced two important Postman techniques: **Basic Auth** as an alternative to bearer tokens, and **path variables** using the `:id` syntax with values supplied through the **Params** tab. The same booking id will be reused in the upcoming PATCH and DELETE assignments.